CYBERSECURITY TRACK • PYTHON MODULE

Assignment 6

Service Banner Grabbing Tool (Python)

Done by: Abdulhadi Al-Ahdal

1. Project Overview

The objective of this assignment is to build an active reconnaissance tool in Python designed to extract service identification banners from open network ports. By establishing direct TCP connections, the application captures greeting signatures sent by target servers or triggers dynamic responses using HTTP probing protocols. This enables security analysts to accurately identify running services, software versions, and operating system attributes across network targets like SSH, FTP, and web servers.

2. Development Steps

1 **TCP Socket Handler**

Establish custom low-level TCP socket client connections targeting specified open ports on the target machine.

2 **Interactive Probing**

Differentiate between passive services sending banners upon connection and interactive web services requiring HTTP request payloads.

3 Response Data Buffer

Receive binary responses up to 1024 bytes and safely decode string formats while discarding invalid character sequences.

4 Output Formatting

Extract and sanitize primary header strings from banner responses to provide a clean visual breakdown of target services.

3. Technical Features & Utilities

Safe String Decoding: Employs `decode('utf-8', errors='ignore')` to process raw byte streams safely without encoding exceptions.

Primary Line Extraction: Uses `splitlines()[0]` to parse and print the concise identifying header signature from server payloads.

4. Implementation Source Code

`banner_grabber.py`

Python 3.x

```
import socket
import sys
import time

def build_http_payload(host):
    payload = f"HEAD / HTTP/1.1\r\nHost: {host}\r\nUser-Agent: ServiceChecker/1.0\r\nAccept: */*\r\nConne
    return payload.encode("utf-8")
```

```
def clean_banner_string(raw_bytes):
    decoded_text = raw_bytes.decode("utf-8", errors="ignore").strip()
    if not decoded_text:
        return "No banner output received"
    lines = decoded_text.splitlines()
    for line in lines:
        cleaned_line = line.strip()
        if cleaned_line:
            return cleaned_line
    return "Empty banner content"

def fetch_banner(ip_address, port, timeout_sec=2.5):
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.settimeout(timeout_sec)

    web_ports = [80, 443, 8000, 8080, 8443]

    try:
        connection_status = sock.connect_ex((ip_address, port))
        if connection_status != 0:
            sock.close()
            return None, "CLOSED"

        if port in web_ports:
            http_request = build_http_payload(ip_address)
            sock.sendall(http_request)

        response_bytes = sock.recv(1024)
        sock.close()

        if response_bytes:
```

```
            banner = clean_banner_string(response_bytes)
            return banner, "OPEN"
        else:
            return "No banner received", "OPEN"

    except socket.timeout:
        sock.close()
        return "Connection timed out", "OPEN"
    except Exception as err:
        sock.close()
        return f"Error encountered: {str(err)}", "ERROR"

def resolve_target_host(target_input):
    try:
        resolved_ip = socket.gethostbyname(target_input)
        return resolved_ip
    except socket.gaierror:
        return None

def run_banner_scanner(target, ports_list):
    resolved_ip = resolve_target_host(target)
    if not resolved_ip:
        print(f"[-] Failed to resolve host: {target}")
        sys.exit(1)

    print("=" * 65)
    print(f" Target Host   : {target} ({resolved_ip})")
    print(f" Probe Count   : {len(ports_list)} ports")
    print(f" Execution Time: {time.strftime('%Y-%m-%d %H:%M:%S')}")
    print("=" * 65)
    print(f"{'PORT':<10} {'STATUS':<12} {'SERVICE BANNER / DETAILS'}")
    print("-" * 65)
```

```
    open_count = 0

    for port in ports_list:
        banner, status = fetch_banner(resolved_ip, port)
        if status == "OPEN":
            open_count += 1
            print(f"{port:<10} {'OPEN':<12} {banner}")
        elif status == "CLOSED":
            print(f"{port:<10} {'CLOSED':<12} ---")
        else:
            print(f"{port:<10} {'ERROR':<12} {banner}")

    print("-" * 65)
    print(f"Scan complete. Found {open_count} open port(s) out of {len(ports_list)} tested.")
    print("=" * 65)

def main():
    if len(sys.argv) > 1:
        target_host = sys.argv[1]
    else:
        target_host = input("Enter target IP address or domain: ").strip()

    if not target_host:
        target_host = "127.0.0.1"

    default_ports = [21, 22, 23, 25, 53, 80, 110, 143, 443, 445, 8080, 8443]

    run_banner_scanner(target_host, default_ports)
```

```
if __name__ == "__main__":
    main()
```

5. Acceptance Criteria

- ✓ **Accurate Banner Extraction**
  Successfully retrieves detailed service versions from open ports disclosing banner info.
- ✓ **Graceful Exception Handling**
  Handles timeouts and non-responsive ports smoothly without raising unhandled runtime errors.